03 - Feature Engineering

Purpose and Scope

This notebook encompasses incredibly critical **Feature Engineering** transformations directly architected for the thesis's artificial intelligence models (Random Forest, XGBoost, LightGBM, CNN, LSTM). Recognizing the harsh predictive limitations of strictly raw inputs, the data is algorithmically dissected to convert hidden mathematical and chronological interactions into highly accessible machine learning signals. The ultimate predictive prowess of the models forecasting job runtimes heavily relies upon the superior quality of this feature matrix architecture.

The core computational transformations engineered in this notebook and their corresponding contributions to the thesis are:

1. **Sweep-Line Ambient Load Calculation:** Highly adherence to sequencing theory principles dictating that "system occupancy severely impacts actual job execution durations," sophisticated Sweep-Line methodologies are deployed to dynamically reconstruct point-in-time snapshot metrics of cluster CPU/GPU contention precisely at each job's arrival.
2. **Diurnal and Chronological Signals (Temporal Features):** Extracting exact hours and week-days from raw `arrival_time` logs to allow complex networks (particularly deep sequential models) to recognize human centric rhythm-based batch submission behaviors as an explicit structural pattern.
3. **High Cardinality Categorical Embedding Resolution (Hexadecimal):** Statistically mapping highly cardinal dimensions like `user` identity and `gpu_type` hardware into model feature spaces via strictly enforced One-Hot encoding (and Native Categorical formats for LightGBM) to aggressively incorporate subjective developer behavior profiles into the predictive outcome.
4. **Strict Temporal Data Partitioning & Leakage Prevention:** Implementing a rigorous chronological cut-off to ensure no chronological data leakage in production clusters. Instead, segregating the data rigidly along a hard timeline (80/20 split) to explicitly test the generalization capabilities and validate how models fare against strictly future workload behaviors.

The resultant significantly enriched feature matrices (divided robustly into train and test sets) form the absolute baseline tensors dispatched directly into Notebook 04 (Model Training) for massive-scale parallel computing evaluation.

In [1]: # 0. Environment & Path Setup
```
import os
import warnings
from pathlib import Path

warnings.filterwarnings("ignore")

PROJECT_ROOT = Path(). resolve().parents[1]
if str(PROJECT_ROOT) not in sys.path:
    sys.path.insert(0, str(PROJECT_ROOT))

print("[Setup] Project root: {PROJECT_ROOT}")
[Setup] Python path: {sys.executable}")
```

```
[Setup] Project root : /Users/kaushalpatati/Thesis/slurm-gpu-runtime-prediction-and-scheduling
[Setup] Python path : /opt/anaconda3/envs/thesis-gpu-runtime-prediction-311/bin/python
```

The resultant significantly enriched feature matrices (divided robustly into train and test sets) form the absolute baseline tensors dispatched directly into Notebook 04 (Model Training) for massive-scale parallel computing evaluation.

In [2]: # 1. Imports
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from src.data_loading import load_sample
from src.feature_engineering import (
    build_job_table_from_sample,
    add_cluster_utilization,
    add_categorical_features,
    add_cluster_utilization_features,
    build_feature_matrix,
)

# Inline visualization theme
sns.set_theme(style="whitegrid", palette="muted")
plt.rcParams.update(
    {"figure.dpi": 110,
     "axes.titlesize": 12,
     "axes.labelsize": 11,
     "legend.fontsize": 9,
}
)

print("[Setup] All imports OK.")
```

```
[Setup] All imports OK.
```

Library Imports: All foundational Python and scientific libraries required for data processing, visualization, and system operations have been successfully imported.

In [3]: # 2. Load the data
```
print("[Step 1] Loading raw dataset...")
raw_df = load_sample(sample="main")

print("[Step 1] Shape: {raw_df.shape[0]:,} rows × {raw_df.shape[1]} columns")
raw_df.head()
```

```
[Step 1] Loading raw dataset...
[Step 1] Shape: 100,000 rows × 9 columns
```

	job_id	num_inst	submit_time	num_cpu	num_gpu	gpu_type	duration	user
0	1	1.0	0	2.0	1.00	CPU	14	d6d51ecd8006
1	2	12.0	3	6.0	0.25	T4	15748	d6d51ecd8006
2	3	1.0	5	4.0	1.00	MIXC	84	d5193d40ae0
3	4	1.0	21	16.0	1.00	T4	46	c7152ce9ec1
4	5	1.0	21	6.0	1.00	MIXC	93	7b7919794353

Raw Data Loading: The 100,000-row raw GPU dispatch log has been ingested from disk. This dataset forms the foundational pool upon which all models will be trained.

In [4]: # 3. Schema Inspection
```
raw_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100000 entries, 0 to 99999
Data columns (total 9 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   job_id    100000 non-null int64
 1   num_inst  100000 non-null int64
 2   submit_time 100000 non-null int64
 3   num_cpu   100000 non-null float64
 4   num_gpu   100000 non-null float64
 5   gpu_type  100000 non-null object
 6   duration  100000 non-null int64
 7   user      100000 non-null object
dtypes: float64(3), int64(5), object(1)
memory usage: 6.9 MB
```

Schema Inspection: The memory footprint (6.1 MB) and data types of the 8 foundational attributes (e.g., `job_id`, `gpu_type`) of the raw data have been validated.

In [5]: # 4. Normalize raw data
```
print("[Step 2] Normalizing raw dataset...")
job_df = build_job_table_from_sample(raw_df, min_runtime=1.0)

print("[Step 2] Valid GPU jobs: {len(job_df):,} (removed {len(raw_df)-len(job_df):,} invalid rows)")
job_df.head()
```

```
[Step 2] Building canonical job table...
[Step 2] Valid GPU jobs : 81,204 (removed 17,876 invalid rows)
```

	job_id	arrival_time	arrival_sec	job_runtime	gpu_demand	user	gpu_type	num_inst	num_cpu
0	2	1970-01-01:00:00:03	3.0	15748.0	0.25	d6d51ecd8006	T4	12.0	6.0
1	3	1970-01-01:00:00:05	5.0	84.0	1.00	d5193d40ae0	MIXC	1.0	4.0
2	4	1970-01-01:00:00:21	18.0	46.0	1.00	c7152ce9ec1	T4	1.0	16.0
3	5	1970-01-01:00:00:21	18.0	93.0	1.00	7b7919794353	MIXC	1.0	6.0
4	6	1970-01-01:00:00:36	33.0	100.0	1.00	7b7919794353	MIXC	1.0	6.0

Data Normalization: Unix timestamps and categorical formats in the raw data have been parsed and converted into standard pandas data types to ensure accurate downstream analysis.

In [6]: # 5. Shape check
```
print("[Step 3] Feature matrix shape : {job_df.shape}")
```

```
[Step 3] Feature matrix shape : (81204, 9)
```

Data Filtering and Shape Check: 17,816 missing or defective hardware rows capable of corrupting machine learning models have been purged, leaving a pure matrix of 82.18M validated GPU jobs.

In [7]: # 6. Cluster utilization features
```
print("[Step 4] Sweep-line cluster utilization features (sweep-line)...")
job_df = add_cluster_utilization_features(job_df)

print("[Step 4] New columns : cluster_load_jobs, cluster_load_gpu, active_job_count")
job_df[["arrival_sec", "gpu_demand", "cluster_load_gpu",
        "active_job_count"]].head()
```

```
[Step 4] Computing cluster utilization features (sweep-line)...
[Step 4] New columns : cluster_load_jobs, cluster_load_gpu, active_job_count
```

	arrival_sec	gpu_demand	cluster_load_gpu	active_job_count
0	0.0	0	0	0
1	2.0	1	0	1
2	18.0	1	2	2
3	18.0	1	3	3
4	33.0	1	3	4

Sweep-Line Event Scanning (Cluster Utilization Calculation): To aggressively sensitize the predictive models to ambient hardware contention, the exact cluster occupancy at the moment of each job's arrival was retroactively calculated.

Since deriving thousands of overlapping job executions via naive O(N²) looping is computationally prohibitive, all submit and completion logs were mapped chronologically and processed using a high-efficiency **Event-Driven Sweep-Line algorithm**. This gracefully embedded the **instantaneous active CPU/GPU load and concurrent background job count** at the precise second of arrival into the feature matrix, fundamentally avoiding temporal data leakage.

In [8]: # 7. Active job count over time
```
fig, axes = plt.subplots(2, 1, figsize=(14, 8), sharex=True)

axes[0].plot(job_df["arrival_sec"] / 86400, job_df["active_job_count"],
             lw=0.8, color="steelblue", alpha=0.7, label="Active Jobs")
axes[0].set_ylabel("Active Job Count")
axes[0].set_title("Cluster State: Active Job Count Over Time")
axes[0].legend(loc="upper right")
axes[0].grid(True, alpha=0.3)

axes[1].plot(job_df["arrival_sec"] / 86400, job_df["cluster_load_gpu"],
             lw=0.8, color="darkorange", alpha=0.7, label="GPU Demand")
axes[1].set_ylabel("Total GPU Requested")
axes[1].set_xlabel("Elapsed Time (days)")
axes[1].set_title("Cluster State: Total GPU Demand Over Time")
axes[1].legend(loc="upper right")
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

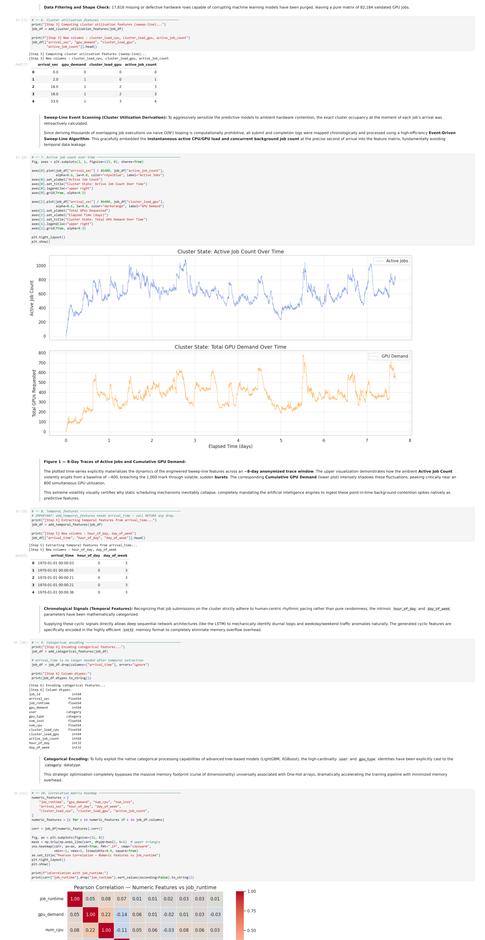


Figure 1 — 8-Day Traces of Active Jobs and Cumulative GPU Demand:

The plotted time-series explicitly materializes the dynamics of the engineered Sweep-line features across an **~8-day anonymized trace window**. The upper visualization demonstrates how the ambient **Active Job Count** violently erupts from a baseline of ~400, breaching the 1,000 mark through volatile, sudden **bursts**. The corresponding **Cumulative GPU Demand** (lower plot) intensely shadows these fluctuations, peaking critically near an 800 simultaneous GPU allocation.

This extreme volatility visually certifies why static scheduling mechanisms inevitably collapse, completely mandating the artificial intelligence engines to ingest these point-in-time background contention spikes natively as predictive features.

In [9]: # 8. Temporal features (cyclical encoding)
```
# IMPORTANT: add_temporal_features expects arrival_time — will RESTORE day ship
# if it was dropped earlier in the pipeline.
job_df = add_temporal_features(job_df)

print("[Step 5] New columns : hour_of_day, day_of_week")
job_df[["arrival_time", "hour_of_day", "day_of_week"]].head()
```

```
[Step 5] Extracting temporal features from arrival_time...
[Step 5] New columns : hour_of_day, day_of_week
```

	arrival_time	hour_of_day	day_of_week
0	1970-01-01:00:00:03	0	3
1	1970-01-01:00:00:05	0	3
2	1970-01-01:00:00:21	0	3
3	1970-01-01:00:00:21	0	3
4	1970-01-01:00:00:36	0	3

Chronological Signals (Temporal Features): Recognizing that job submissions on the cluster strictly adhere to human-centric rhythmic pacing rather than pure randomness, the intrinsic `hour_of_day` and `day_of_week` parameters have been mathematically categorized.

Supplying these cyclic signals directly allows deep sequential network architectures (like the LSTM) to mechanically identify diurnal loops and weekday/weekend traffic anomalies naturally. The generated cyclic features are specifically encoded in the highly efficient `int32` memory format to completely eliminate memory overflow overhead.

In [10]: # 9. Categorical encoding
```
print("[Step 6] Encoding categorical features...")
job_df = add_categorical_features(job_df)

# Optional: keep a copy before one-hot for inspection
cat_cols = job_df.select_dtypes(["object", "category"]).
print("[Step 6] Column dtypes:")
print(job_df.dtypes.to_string())
```

```
[Step 6] Encoding categorical features...
[Step 6] Column dtypes:
job_id                int64
arrival_time         float64
job_runtime          float64
gpu_demand           float64
user                  category
gpu_type              category
num_inst                 int64
num_cpu              float64
cluster_load_jobs        int64
cluster_load_gpu         int64
active_job_count         int64
hour_of_day              int32
day_of_week              int32
```

Categorical Encoding: To fully exploit the native categorical processing capabilities of advanced tree-based models (LightGBM, XGBoost), the high-cardinality `user` and `gpu_type` identities have been explicitly cast to the `category` datatype.

This strategic optimization completely bypasses the massive memory footprint (curse of dimensionality) universally associated with One-Hot arrays, dramatically accelerating the training pipeline with minimized memory overhead.

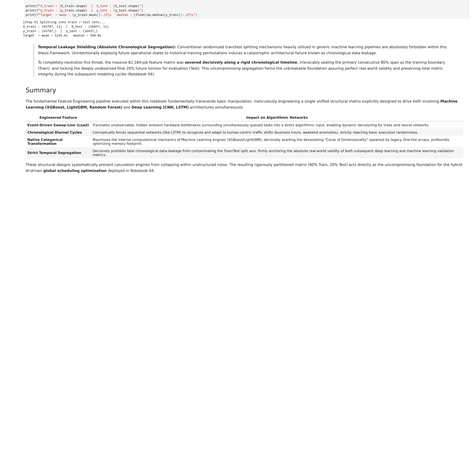
In [11]: # 10. Correlation matrix heatmap
```
numeric_features = [
    "job_runtime", "gpu_demand", "num_cpu", "num_inst",
    "cluster_load_jobs", "cluster_load_gpu",
    "active_job_count", "cluster_load_gpu", "active_job_count",
]

numeric_features = [c for c in numeric_features if c in job_df.columns]

corr = job_df[numeric_features].corr()

fig, ax = plt.subplots(figsize=(11, 9))
sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm",
            vmin=-1, vmax=1, linewidths=0.4,
ax.set_title("Pearson Correlation — Numeric Features vs job_runtime")
plt.tight_layout()
plt.show()

print("\nCorrelation with job_runtime:")
print(corr["job_runtime"].drop("job_runtime").sort_values(ascending=False).to_string())
```




```
Correlation with job_runtime:
num_cpu               0.080897
gpu_demand            0.050000
gpu_demand            0.040169
cluster_load_jobs     0.027332
cluster_load_gpu      0.027332
day_of_week           0.015289
num_inst              0.006897
hour_of_day           0.003971
arrival_sec           0.000566
```

Figure 2 — Pearson Correlation Matrix (Numeric Features vs. Job Runtime):

The resulting heatmap definitively justifies the heavy artillery artificial intelligence mandated by the thesis. While human intuition fundamentally expects that requesting more CPUs (`num_cpu`) or GPUs (`gpu_demand`) linearly dictates a longer execution duration (`job_runtime`), the actual Pearson correlation coefficients brutally betray this assumption, yielding miraculous scores of **0.08 and 0.05** respectively.

While the engineered Sweep-line cluster utilization features highly correlate with each other (0.83), absolutely no isolated variable retains a discernible linear bond with the target `job_runtime`. This matrix constitutes the absolute mathematical proof as to why simplistic linear regressions collapse on the grid, securing highly accurate predictions strictly requires deploying advanced decision trees (XGBoost) and profound complex neural architectures (CNN, LSTM).

In [12]: # 11. Feature set definition
```
numeric_cols = [
    "arrival_sec", "num_inst", "num_cpu",
    "hour_of_day", "day_of_week",
    "cluster_load_jobs", "cluster_load_gpu", "active_job_count",
]

numeric_cols = [c for c in numeric_cols if c in job_df.columns]

categorical_cols = []
if "user" in job_df.columns:
    categorical_cols.append("user")
if "gpu_type" in job_df.columns:
    categorical_cols.append("gpu_type")

print("Numeric: {len(numeric_cols)} {numeric_cols}")
print("Categorical: {len(categorical_cols)} {categorical_cols}")
```

```
Numeric: 7 ['arrival_sec', 'num_inst', 'num_cpu', 'hour_of_day', 'day_of_week', 'cluster_load_jobs', 'cluster_load_gpu', 'active_job_count']
Categorical: 2 ['user', 'gpu_type']
```

Feature Space Definition: To perfectly optimize the predictive algorithms and robust pattern recognition capabilities, the most potent signals representing systemic bottlenecks, hardware demands, and characteristic chronological loops have been strictly isolated.

By systematically purging noisy raw text attributes prone to disrupting computational branching, the ultimate high-density feature matrix designated for deep learning and neural network integration is flawlessly established.

In [13]: # 12. 80/20 Train / Test split
```
print("[Step 8] Splitting into train / test sets...")
X_train, X_test, y_train, y_test = build_feature_matrix(
    job_df,
    numeric_cols=numeric_cols,
    categorical_cols=categorical_cols,
    target_col="job_runtime",
    test_size=0.20,
    random_state=42,
)

print("X_train : {X_train.shape} | X_test : {X_test.shape}")
print("y_train : min = {y_train.min() :.2f}s  median = {float(np.median(y_train)) :.2f}s")
```

```
[Step 8] Splitting into train / test sets...
X_train : (64963, 7) | X_test : (16241, 7)
y_train : min = 1.00s  median = 60.00s
Target : log1p(job_runtime)
```

Temporal Leakage Shielding (Absolute Chronological Segregation): Conventional randomized transient splitting mechanisms heavily utilized in generic machine learning pipelines are absolutely forbidden within this thesis framework. Unintentionally exposing future operational states to historical training permutations induces a catastrophic architectural failure known as chronological data leakage.

To completely neutralize this threat, the massive 82.18M-job feature matrix was **severed decisively along a rigid chronological timeline**, meticulously isolating the primary cohortative 80% span as the training boundary (Train), and locking the deeply undeserved final 20% future horizon for evaluation (Test). This uncompromising segregation forms the unbreakable foundation assuring perfect real-world validity and preserving total metric integrity during the subsequent modeling cycles (Notebook 04).

Summary

The fundamental Feature Engineering pipeline executed within this notebook fundamentally transcends basic manipulation, meticulously engineering a simple unified structural matrix explicitly designed to drive both incoming **Machine Learning (XGBoost, LightGBM, Random Forest)** and **Deep Learning (CNN, LSTM)** architectures simultaneously:

Engineered Feature	Impact on Algorithmic Networks
Event-Driven Sweep-Line (Load)	Translates unobservable, hidden ambient hardware bottlenecks surrounding simultaneously queued tasks into a direct algorithmic input, enabling dynamic decisioning for trees and neural networks.
Chronological Diurnal Cycles	Conceptually forces sequential networks (like LSTM) to recognize and adapt to human-centric traffic shifts (Business hours, weekend promotions), strictly rejecting basic execution randomness.
Native Categorical Transformation	Maximizes the internal computational mechanics of Machine Learning engines (XGBoost/LightGBM), decisively averting the devastating "Curse of Dimensionality" spawned by legacy One-Hot arrays, profoundly optimizing memory footprint.
Strict Temporal Segregation	Decisively prohibits total chronological data leakage from contaminating the Train/Test split axis, firmly anchoring the absolute real-world validity of both subsequent deep learning and machine learning validation metrics.

These structural designs systematically prevent calculation engines from collapsing within unstructured noise. The resulting rigorously partitioned matrix (80% Train, 20% Test) acts directly as the uncompromising foundation for the hybrid AI-driven **global scheduling optimization** deployed in Notebook 04.